

M2 - Architecture et Programmation d'accélérateurs Matériels.

(APM 2024-2025)



TP2

Asynchronisme – Utilisation des Streams

julien.jaeger@cea.fr
lucas.vanlanker@cea.fr

Les objectifs de ce TP sont :

- Utilisation des Streams CUDA pour faire du calcul asynchrone.
- Permettre le recouvrement des transferts mémoire par du calcul.

I Portage sur GPU (Stream par défaut)

Le code source principal se situe dans la section CODE du présent TP. Le programme représente le calcul d'une fonction exponentielle sur un espace 1D.

Q.1: Portez la fonction "func" sur GPU, en configurant une grille 1D qui contient suffisamment de blocs et de threads pour couvrir l'ensemble du vecteur.

II Initialisation des Streams

Un flux (ou Stream) CUDA est une suite d'opérations qui sont exécutées selon l'ordre dans lequel elles sont appelées sur le GPU. Ce modèle de programmation est utilisé pour introduire de la concurrence sur le GPU :

- Les opérations CUDA venant de différents flux peuvent s'exécuter de manière concurrentes.
- Les opérations CUDA entre différents flux peuvent être interchangeables.

Nous allons dans un premier temps créer et initialiser différents flux à partir du code réalisé lors de la question précédente.

Q.2: Allouez un ensemble `NSTREAMS` de Streams CUDA et initialisez-les.

Q.3: Sans spécifier de Stream sur lequel exécuter un kernel CUDA, quel stream est utilisé ? Quelle particularité a-t-il ?

Correction

Le Stream par défaut est utilisé. Toutes les opérations CUDA sont synchrones sur ce flux, à l'exception de certaines opérations, telles que les lancements de kernels.

Q.4: Modifiez le lancement du kernel CUDA pour que `NSTREAMS` instances de ce kernel soient exécutées sur des Streams différents.

Q.5: Qu'observez-vous au niveau des performances ? Pourquoi et comment les résoudre ?

Correction

Bien que les kernels s'exécutent de manière concurrente, les copies de données vers et depuis le device se font d'un seul bloc synchrone. Pour améliorer les performances, les copies doivent être asynchrones, il faut donc utiliser la fonction `cudaMemcpyAsync`, elles pourront alors être recouvertes par du calcul.

Q.6: Utilisez la fonction `cudaMemcpyAsync` pour effectuer les copies mémoires vers et depuis le GPU. Quel effet a cette fonction ?

NB : N'oubliez pas de synchroniser le host et le device avant de faire la vérification du résultat avec `cudaDeviceSynchronize` pour éviter toute corruption mémoire.

Q.7: À quoi sert la fonction `cudaMallocHost` ? Modifiez votre code en conséquence.

Correction

La copie asynchrone utilise un mécanisme DMA (Direct Memory Access). Le GPU a alors connaissance de l'adresse physique du tableau sur l'hôte. Par conséquent, celui-ci ne doit pas migrer la page physique au cours de son utilisation. D'où l'utilisation de mémoire punaisée (*pinned memory*). `cudaMallocHost` permet d'allouer de la mémoire punaisée dans la mémoire de l'hôte. L'appel système `malloc` ne permet pas d'allouer ce type de mémoire et est à proscrire lors de l'utilisation de copie asynchrone entre l'hôte et le GPU.

Q.8: Classer les fonctions et les appels de kernels en deux catégories : les synchrones et les asynchrones.

Correction

appel synchrone	appel asynchrone
<code>cudaMalloc</code> , <code>cudaFree</code>	<code>cudaMallocAsync</code>
<code>cudaMallocHost</code> , <code>cudaFreeHost</code>	<code>cudaFreeAsync</code>
<code>cudaStreamCreate</code> , <code>cudaStreamDestroy</code>	<code>cudaMemcpyAsync</code>
<code>cudaDeviceSynchronize</code>	<code>MyKernel</code>

Q.9: En fixant `NSTREAMS = 2`, dessiner un schéma représentant les différentes tâches sur les flux d'exécution ainsi que leur dépendances.

Nous prendrons comme notation :

$\mathbf{A} \rightarrow \mathbf{B}$: l'exécution de B ne peut commencer tant que celle de A n'est pas terminée.

$\mathbf{A} \Rightarrow \mathbf{B}$: l'exécution de B ne peut se terminer tant que celle de A n'est pas terminée.

Liste des tâches :

Création S0	Création S1	Malloc Host	Thread Sync
requête cpy H→D [0,N[	requête cpy H→D [N,2N[	requête cpy D→H [0,N[	requête cpy D→H [N,2N[
cpy H→D [0,N[	cpy H→D [N,2N[	cpy D→H [0,N[	cpy D→H [N,2N[
requête K1 [0,N[	requête K1 [N,2N[	exécution K1 [0,N[	exécution K1 [N,2N[

Correction

voir graph.pdf où la flèche verte correspond au $A \Rightarrow B$.

III Mesure de temps

Q.10: Utiliser la fonction système `gettimeofday` pour mesurer le temps passé dans l'exécution du programme CUDA.

Correction

Voir le programme `stream_ex1_gettimeofday.cu` pour la correction.

Q.11: Avec les fonctions `gettimeofday` et `cudaDeviceSynchronize` mesurer le temps total d'exécution des kernels uniquement (sans compter le temps des transferts mémoire, etc...). Ici, on demande le temps total, pas le temps individuel des kernels

Correction

Voir le programme `stream_ex1_kernels_gettimeofday.cu` pour la correction.

Q.12: Quel est l'impact (inconvenient) sur l'exécution générale du programme de cette dernière façon de mesurer ? Vous pouvez vous aider du graphe de dépendance précédent et l'enrichir.

Correction

Les appels à `cudaDeviceSynchronize` imposent 2 points de synchronisation entre les streams introduisant ainsi une sérialisation des étapes : on effectue d'abord toutes les copies sur tous les streams, puis on exécute tous les kernels et enfin on termine par toutes les copies. On perd l'intérêt d'utiliser les copies asynchrones et l'exécution concurrente des kernels.

Q.13: Comment mesurer le temps de chacun des kernels avec la technique précédente ?

Correction

Si l'on ne dispose que de `cudaDeviceSynchronize` et de `gettimeofday` pour mesurer le temps de chaque kernel de chaque stream alors on est obligé d'exécuter les streams les uns après les autres en forçant une synchronisation et un relevé de temps entre chaque stream. Il n'y a plus d'exécution concurrente entre kernels : ils s'exécutent les uns après les autres. Voir fichier `stream_ex1_onekernel_gettimeofday.cu`.

Q.14: Comment utiliser les événements (fonctions `cudaEvent*`) pour mesurer le temps de chaque kernel et le temps total de tous les kernels ? Commentaire par rapport à la façon précédente ?

Correction

Les appels à `cudaEventRecord` permet de positionner les événements dans le flux séquentiel d'un stream. On peut ainsi marquer le début et la fin de l'exécution d'un kernel au sein d'un stream sans synchroniser les streams entre eux. La fonction `cudaEventSynchronize` permet d'attendre l'exécution réelle d'un événement. Voir fichier *stream_ex1_events.cu*.

IV Multiplication Matricielle

Q.15: Reprenez l'exercice de multiplication matricielle du TP1 et transformez le code pour le rendre multi-streams. Dans ce cas, un stream s'occupera d'un paquet de lignes à traiter. Essayez d'abord avec le kernel "coalescé" puis avec le kernel de multiplication par tuiles.

Quels sont vos gains de performances par rapport à la version synchrone ?

Correction

Voir `dgemm_async.cu`
Sur un GPU Quadro P5000 (Driver 560.35.03 et CUDA 12.6), avec `N = 4096` et `N_STREAMS = 4` :
Pour des FP64, la version coalescée se fait en 0.80s, soit 172 GFLOP/s.
La version par tuiles se fait en 1.02s, soit 135 GFLOP/s, donc environ 1.5 fois plus rapide que la version synchrone.
Pour des FP32, la version coalescée se fait en 0.50s, soit 273 GFLOP/s.
La version par tuiles en 0.18s, soit 760 GFLOP/s, donc environ 1.3 fois plus rapide que la version synchrone.